Learning Verified Monitors for Hidden Markov Models*

Luko van der Maas and Sebastian Junges

Radboud University, Nijmegen, the Netherlands
{luko.vandermaas,sebastian.junges}@ru.nl

Abstract. Runtime monitors assess whether a system is in an unsafe state based on a stream of observations. We study the problem where the system is subject to probabilistic uncertainty and described by a hidden Markov model. A stream of observations is then unsafe if the probability of being in an unsafe state is above a threshold. A correct monitor recognizes the set of unsafe observations. The key contribution of this paper is the first correct-by-construction synthesis method for such monitors, represented as finite automata. The contribution combines four ingredients: First, we establish the coNP-hardness of checking whether an automaton is a correct monitor, i.e., a monitor without misclassifications. Second, we provide a reduction that reformulates the search for misclassifications into a standard probabilistic system synthesis problem. Third, we integrate the verification routine into an active automata learning routine to synthesize correct monitors. Fourth, we provide a prototypical implementation that shows the feasibility and limitations of the approach on a series of benchmarks.

1 Introduction

Runtime assurance is an essential ingredient in the deployment of safe autonomous systems [18,41]. Runtime monitors provide assurance by flagging potentially dangerous system behavior, based on a system execution. More precisely, a monitor receives a stream of observations about the system and outputs a verdict, e.g., it raises an alarm that the system has left some safety envelope. A monitor is correct if it correctly raises such alarms based on a formal specification. Various challenges in creating correct runtime monitors for (semi-)autonomous systems have been identified [41], such as: (1) the state of the system is only partially observable, i.e., the stream of observations comes from sensor readings and does not uniquely identify the state of a system, (2) the behavior of the system and/or the sensors may be subject to probabilistic uncertainty, (3) the monitor itself is subject to resource constraints (time, memory, etc.), and (4) the monitor is itself safety-critical and should therefore be subject to extensive validation. Challenges (1,2) can be addressed by modelling the system as a hidden Markov model (HMM), and Challenges (3,4) can be addressed by representing a monitor

* This work has been partially funded by the NWO grant FuRoRe (OCENW.M.22.282).

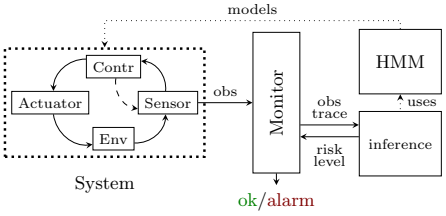


Fig. 1: White-box monitoring of systems

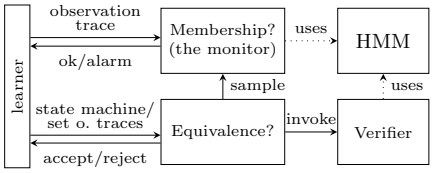


Fig. 2: Learning monitors.

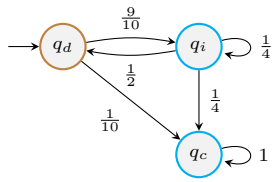
as, e.g., a (small) finite automaton. Concretely, this paper focuses on the following question: *Is a given finite automaton a correct monitor for a given and known HMM?* This paper studies the complexity of this problem, provides a practical verification approach, and embeds it into a framework to learn monitors.

What are HMMs? In this paper, we assume that the system including its sensors is adequately modeled as a discrete Hidden Markov Model (HMM) [39] and that we have full access to this HMM. Markov chains (MCs) describe system behavior subject to probabilistic uncertainty. Paths through an MC are sequences of states that describe system executions. HMMs extend MCs by labelling their states with *observations*. Intuitively, the observations can be used to model the information that the monitor receives in every state. In HMMs, every path can be lifted to a sequence of observations, which we call a *trace*. The trace associated to a system execution describes the information received by the monitor.

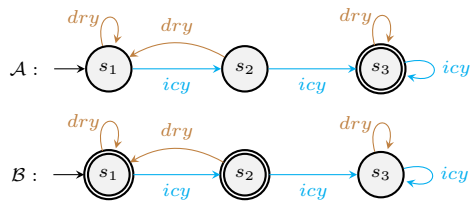
Monitoring with HMMs. Monitoring with HMMs assumes that a monitor receives a trace from the system and performs inference on the HMM modelling the system (see Fig. 1). In the inference step, the key task is to estimate whether the current system state is dangerous, based on the available information in the form of a trace. Intuitively, the *risk of a trace* [34] quantifies how likely it is that the system state is dangerous. Formally, this can be defined as the probability of ending in a dangerous state, conditioned on the fact that the system execution matches the trace. For a given trace, we may compute this risk, e.g., either via model checking [16] or by a (forward) filtering that tracks a distribution over the current states [34,39]. We call a trace *unsafe* if its risk exceeds an acceptable threshold. Monitors should raise alarms only for unsafe traces.

What are correct monitors? Like in [1], we summarize the behavior of monitors by the set of traces (i.e., a formal language) on which they raise an alarm. A monitor is correct iff it raises an alarm on all unsafe traces. We highlight that a monitor can be correct without doing inference at run time [23]! The key verification problem in this paper asks whether a monitor, represented as a deterministic finite automaton, accepts (i.e., raises an alarm on) exactly the unsafe traces. In this paper, we only consider traces that are bounded by some fixed horizon.

Illustrative toy example. As a running example, we consider an oversimplified car driving scenario, loosely inspired by runtime monitors obtained from high-fidelity simulations [46]. A car can be in three states: It can be on a dry road, on an icy patch, or it has drifted off the road. The HMM in Fig. 3a describes how



(a) HMM. States q_i, q_c have the observation *icy*, state q_d has *dry*. In state q_c the vehicle is off the road.



(b) A correct monitor $\mathcal{A}$ and an incorrect monitor $\mathcal{B}$ (for $\lambda = 0.25$ and $h = 3$).

Fig. 3: Running example: HMM (a) and two monitors (b).

a car alternates between dry and icy road segments, and where being on an icy (dry) segment positively affects the probability that the next segment is icy (dry). When on an icy road, there is a higher probability to drift off the road. Our sensor can detect dry roads, but cannot distinguish between icy roads and off-road conditions. For a trace τ such as $dry \cdot icy \cdot icy$, we can define the trace risk as the probability to be off-road conditioned on τ, $^{13}/_{22}$ (for this trace, see example 2). In Fig. 3b, monitor $\mathcal{A}$ is a finite automaton that describes the set of traces that end with two consecutive icy patches. The verification question in this paper is now: *Is this a correct monitor for threshold* 0.25*?* That is, is an alarm raised iff the trace risk is above 0.25?

Model-based approach. In this paper, we assume we are given a *useful* model of the system. While such white-box methods can be efficient and explainable, they require a model of the system, which is not always realistic. Alternatively, model-free approaches [20] require no such assumptions, but they have weaker guarantees. Black-box methods [13] fit in between, they learn a model of the system and then use a white-box algorithm to provide the monitor. While we focus on the white-box approach, the results thus contribute towards black-box settings.

Computational complexity results. Deciding whether a monitor is correct is coNP-complete (Thm. 4), assuming that the horizon is unary encoded. In particular, we study the dual problem that asks whether a monitor misclassifies at least one trace as safe/unsafe. A simpler variant of this problem asks whether there is a bounded trace in the HMM that is unsafe. This problem is already strongly NP-hard (Lem. 5) and the optimization problem that asks to compute (the risk of) most risky trace is APX-hard (Lem. 6), which indicates that it is intractable to even approximate this risk.

Our verification approach. While deciding whether a monitor is correct is in general not tractable, we suggest utilizing recent advances in the synthesis of probabilistic systems [9]. Our presentation focusses on proving the absence of *missed alarms*, i.e., we concentrate on showing that the monitor correctly identifies every unsafe trace, but we show that a similar reduction can be used to show that a monitor correctly identifies the safe traces. First, for a single trace, computing the risk can be reduced to computing a reachability probability in an MC that

is some kind of product between a deterministic finite automata (DFA) that accepts exactly the trace and the HMM [34]. Thus, inspired by [14], we reduce our problem to the question: *Is there a DFA (accepting exactly one trace, accepted by our monitor) such that the probability in the product MC exceeds a threshold?* The answer is no iff the monitor has no missed alarms. We formalize the problem using colored MDPs and solve it using (exact) probabilistic model synthesis methods [8], as implemented in the tool PAYNT [9].

Our learning approach. Ultimately, we do not only want to verify monitors, but we want to *synthesize* correct monitors. We already mentioned that the monitors are formal languages. When considering bounded traces, these are regular, and can thus be captured using DFAs. Thus, we aim to synthesize DFAs. We do this using active automata learning (AAL, [11,47]), which is similar to other oracle-guided inductive synthesis loops [32]. Consider Fig. 2: To use AAL, we must provide a *membership oracle* that decides for a single trace whether the risk of the trace exceeds a threshold and an *equivalence oracle* that decides whether a given hypothesis monitor is indeed correct. The membership oracle can be implemented via inference for standard monitors on HMMs (see above), while the equivalence oracle can be implemented by verifying the correctness of the monitor, as popularized in *black-box checking* [37]. Practically, some minor modifications are necessary to embed our approach in an AAL framework: We have to handle the finite horizon, ignore traces that cannot occur, and combine our equivalence oracle with a conformance oracle to boost performance.

Contributions. In summary, this paper provides a first framework to learn runtime monitors—encoded as finite automata—that are verified to be correct with respect to a given HMM. The main contributions are: **(1)** We solve the verification problem by exhaustively searching for a counterexample using probabilistic system synthesis (Sec. 3). **(2)** We learn monitors by using the verifier above to answer equivalence queries in conjunction with active automata learning (Sec. 4). **(3)** We prove hardness of the verification problem (Sec. 5). **(4)** We demonstrate the feasibility and limitations of both the verifier and the learner on a series of benchmarks (Sec. 6). Our prototype finds monitors that are provably correct on 10^{22} traces by verifying HMMs with thousands of states, up to a hundred observations, and monitors with over 100 states.

The extended edition [35, App. A and B] contains proof (sketches) for all lemmas and theorems.

2 Formal Problem Statement

Let X be a finite set. A distribution μ over X is a mapping $\mu\colon X \to [0,1]$ such that $\sum_{x \in X} \mu(x) = 1$. The set of all distributions over X is written $\Delta(X)$.

DFAs. A *deterministic finite automaton* (DFA) is a tuple $\mathcal{A} := (Q, \Sigma, \delta, \iota, F)$. Q is a finite set of *states*, Σ is an *alphabet*, $\delta\colon Q \times \Sigma \rightharpoonup Q$ is a (partial) *transition function*, ι is the *initial state*, and $F \subseteq Q$ is a set of *accepting* states[1]. Let

[1] F is named after the synonymous final states to avoid confusion with actions.

ε denote the empty word. We lift the transition relation to words, which is defined recursively as: $\delta^*(q, \varepsilon) := \varepsilon$ if $a \in \Sigma$ and $\delta^*(q, w \cdot a) := \delta(\delta^*(q, w), a)$. The *language* $\mathcal{L}(\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words that end in a final state of $\mathcal{A}$, $\mathcal{L}(\mathcal{A}) := \{w \in \Sigma^* \mid \delta^*(\iota, w) \in F\}$. We say that $\mathcal{A}$ accepts w if $w \in \mathcal{L}(\mathcal{A})$.

2.1 Models

We introduce MDPs and HMMs: The former are integral to our approach and the latter are crucial to the problem statement. Details can be found in [15].

Definition 1 (MDP). *A* Markov decision process *(MDP) is a tuple* $(S, \iota, Act, \mathbf{P})$ *with a countable nonempty set* S *of* states*, the* initial state $\iota \in S$*, and the* partial transition function $\mathbf{P} \colon S \times Act \rightharpoonup \Delta(S)$*.*

We use $Act(s) := \{a \mid \mathbf{P}(s, a) \neq \bot\}$ as the set of *enabled actions*. We assume no deadlocks, i.e., for every $s \in S$, $Act(s) \neq \emptyset$. A path π is a (possibly infinite) sequence $s_0 \cdot a_0 \cdots \in S \times ((S \times Act)^* \cup (S \times Act)^\omega)$, such that $a_i \in Act(s_i)$ and $P(s_i, a_i)(s_{i+1}) > 0$ for every $i \geq 0$. The last state of a finite path is denoted by $\pi_\downarrow$. The set of paths in MDP $\mathcal{M}$ is denoted as $\Pi^{\mathcal{M}}$, the set of finite paths is $\Pi^{\mathcal{M}}_{fin}$, the set of paths of at most length h is $\Pi^{\mathcal{M}}_{h}$, and the set of paths of exactly length h are $\Pi^{\mathcal{M}}_{=h}$. We write $\mathbf{P}(s, a, s')$ for $\mathbf{P}(s, a)(s')$.

A *Markov chain* (MC) is an MDP where $|Act(s)| = 1$ for every state $s \in S$. We simplify notation and write MCs as a tuple $(S, \iota, \mathbf{P})$, $\mathbf{P}(s)$ to refer to the unique distribution $\mathbf{P}(s, a)$, and $\mathbf{P}(s, s')$ for $\mathbf{P}(s)(s')$. Paths in an MC are sequences of (only) states. As in [15], the probability measure $Pr^{\mathcal{M}}$ of an MC $\mathcal{M}$ is the unique probability measure following from the canonical σ-algebra associated with $\mathcal{M}$. A reachability property on target states T is the set of paths which contain a state $t \in T$. The reachability probability $Pr(\Diamond T)$ for $\Diamond T$ is defined using the standard cylinder set construction.

Definition 2 (HMMs). *A (risk-labelled) HMM is a tuple* $(S, \iota, \mathbf{P}, Z, obs, r)$ *such that* $(S, \iota, \mathbf{P})$ *is an MC,* Z *is a finite set of* observations*,* $obs \colon S \to Z$ *is the (deterministic)* observation function[2]*, and* $r \colon S \to \mathbb{R}_{\geq 0}$ *is the* risk function*.*

Notions such as paths are lifted from MCs. Furthermore, a *trace* τ is a sequence of observations. We lift *obs* from states to paths. We define $Pr(\tau \mid \pi) := 1$ if $obs(\pi) = \tau$ and zero otherwise. The probability of a trace τ is $\sum_{\pi \in \Pi^{\mathcal{M}}} Pr(\pi) \cdot Pr(\tau \mid \pi)$. Finally, the conditional probability on a trace $\tau \in \mathcal{L}(\mathcal{M})$ is defined using Bayes' rule $Pr(\pi \mid \tau) := {}^{Pr(\tau \mid \pi) \cdot Pr(\pi)}/_{Pr(\tau)}$. We define $\mathcal{L}(\mathcal{M}) := \{obs(\pi) \mid \pi \in \Pi^{\mathcal{M}}\}$.

Example 1. We consider the HMM from Fig. 3a and $\tau = dry \cdot icy \cdot icy$. The conditional probability $Pr(q_d \cdot q_i \cdot q_i \mid \tau)$ is ${}^{Pr(\tau \mid q_d \cdot q_i \cdot q_i) \cdot Pr(q_d \cdot q_i \cdot q_i)}/_{Pr(\tau)}$. $Pr(\tau \mid q_d \cdot q_i \cdot q_i)$ is 1, and $Pr(q_d \cdot q_i \cdot q_i)$ is ${}^{9}/_{40}$. The sum of the probabilities of all paths which observe τ is ${}^{11}/_{20}$. Thus, $Pr(q_d \cdot q_i \cdot q_i \mid \tau)$ is ${}^{9}/_{22}$.

[2] We use deterministic observation functions for concise definitions. Stochastic observation functions can be expressed via a blow-up of the HMM, see e.g., [34].

2.2 Formal Problem Statement

Definition 3 (Monitor). *A DFA $\mathcal{A}$ is a monitor for HMM $\mathcal{M}$ if the alphabet for $\mathcal{A}$ coincides with the observations in $\mathcal{M}$.*

Monitors should accept unsafe traces, which we define via their risk (level) [34]:

Definition 4 (Trace risk, safe/unsafe traces). *Given HMM $\mathcal{M}$, the* risk *of $\tau \in \mathcal{L}(\mathcal{M})$ is:*

$$R(\tau) := \sum_{\pi \in \Pi^{\mathcal{M}}_{|\tau|}} Pr(\pi \mid \tau) \cdot r(\pi_{\downarrow}).$$

Let $\lambda_s \leq \lambda_u \in \mathbb{R}_{\geq 0}$ be the safe threshold *and the* unsafe threshold*, respectively. A trace $\tau \in \mathcal{L}(\mathcal{M})$ with $R(\tau) > \lambda_u$ is* unsafe*, while τ is* safe *if $R(\tau) < \lambda_s$.*

We deliberately do not require $\lambda_s = \lambda_u$. By picking $\lambda_s < \lambda_u$, some traces are neither safe nor unsafe, also called inconclusive. We write $\mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ (and $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u}$) for the set of safe (and unsafe) traces of length at most h.

Example 2. We consider the HMM from Fig. 3a, with the risk function assigning 1 to q_c and 0 to all other states. Taking the trace $dry \cdot icy \cdot icy$, there are three paths which could generate this trace. Two paths end in q_c, one ends in q_i. The paths ending in q_c have a conditional probability of $13/22$. Since only these paths have a non-zero risk, the risk of the trace is $13/22 \cdot r(q_c) = 13/22$.

Definition 5 (Missed/False alarms). *Given a monitor $\mathcal{A}$ for HMM $\mathcal{M}$, a horizon h, and thresholds $\lambda_s \leq \lambda_u \in \mathbb{R}_{\geq 0}$, the set of* missed alarms *is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) := \mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \setminus \mathcal{L}(\mathcal{A})$. The set of* false alarms *is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s} \cap \mathcal{L}(\mathcal{A})$.*

Definition 6 (Correct monitor). *Given thresholds λ_s, λ_u and horizon h, a monitor $\mathcal{A}$ for HMM $\mathcal{M}$ is* correct *if $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) = \emptyset = \mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A})$.*

A correct monitor raises an alarm for all unsafe traces and for no safe trace, i.e., missed alarms are false negatives, while false alarms are false positives.

Corollary 1. *A monitor $\mathcal{A}$ is correct iff $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \subseteq \mathcal{L}(\mathcal{A}) \subseteq \Sigma^* \setminus \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$.*

Problem statements. Given HMM $\mathcal{M}$, thresholds λ_s, λ_u and horizon h:

1. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there missed alarms, i.e., is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) = \emptyset$?
2. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there false alarms, i.e., is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset$?
3. Find a correct monitor $\mathcal{A}$ for $\mathcal{M}$ w.r.t. λ_s, λ_u and h.

Problems 1 and 2 together allow checking whether a monitor is correct. Furthermore, a correct monitor must exist, as $\mathbb{U}^{\leq h}$ is finite and thus regular.

Example 3. We discuss monitor correctness for the example from Fig. 3 using the correct monitor $\mathcal{A}$. Given the risk function assigning 1 to q_c and 0 to all other states, the traces $\tau_1 = dry \cdot icy \cdot icy$ and $\tau_2 = dry \cdot icy$ have risks $13/22, 1/10$ respectively. If λ_s is $1/4$ and the horizon h is 3, τ_2 is the trace with maximum risk not accepted by the monitor. Given that its risk is below λ_s, the monitor does not have any missed alarms. Similarly, monitor $\mathcal{A}$ does not have any false alarms for $\lambda_u = 1/4$. Thus, $\mathcal{A}$ is a correct monitor for $\mathcal{M}$ with h, λ_s, and λ_u.

3 Monitor Verification

We present our approach to the monitor correctness problem, which reduces checking the existence of missed alarms to the well-studied policy synthesis problem on colored MDPs, defined below. We first formalize this policy synthesis problem and then present the step-wise transformation. Here, we focus on showing that there are no missed alarms of exactly the length of the horizon (an adaption of Prob. 1). At the end of the section, we generalize our construction to traces of length *at most* the horizon (Prob. 1) and to finding false alarms (Prob. 2).

3.1 Relating Missed Alarms to Color-Consistent Policies

A (memoryless) *policy* for an MDP $\mathcal{M}$ is a function $\sigma\colon S \to Act$, which selects actions for every state. An MDP and a policy induce an MC by only keeping the state-action pairs in the transition function selected by the policy. We denote the probability measure on the MC induced by a policy σ of the MDP $\mathcal{M}$ as $Pr_\sigma^{\mathcal{M}}$ Policy synthesis for an MDP of a (reachability) property ϕ entails finding a policy for an MDP such that the induced MC entails ϕ. Colored (aka: labelled) MDPs [10] are an extension to MDPs that allow expressing dependencies between states that policies must adhere to. The following definition suffices for our needs:

Definition 7 (Colored MDP). *Given an MDP $\mathcal{M}$ with states S, a* colored MDP *is a tuple $\mathcal{M}^C := (\mathcal{M}, C, c)$, where C is a set of* colors*, and $c\colon S \to C$.*

Definition 8 (Color consistent). *A memoryless policy σ for a colored MDP $\mathcal{M}^C$ is* color consistent[3] *if for states s, s', $c(s) = c(s')$ implies $\sigma(s) = \sigma(s')$.*

The set of all color-consistent policies is denoted Σ_c. Policy synthesis for colored MDPs asks to find a color-consistent policy such that the reachability probability to a set of target states is above a certain threshold, or to report that no such policy exists. This problem is NP-hard [22], but efficient heuristics exist [9]. Policy synthesis for colored MDPs can prove the absence of missed alarms:

Theorem 1. *Given an HMM $\mathcal{M}$, monitor $\mathcal{A}$, unsafe threshold λ_u, and horizon h, there is a colored MDP $\mathcal{M}^C$ with target state T and threshold λ s.t.*

$$\exists \sigma \in \Sigma_c.\ Pr_\sigma^{\mathcal{M}^C}(\Diamond T) \geq \lambda \quad \textit{iff} \quad \exists \tau \in \mathsf{mA}^{=h}_{\mathcal{M},\lambda_u}(\mathcal{A}).$$

Our proof, outlined in this section, is constructive and we show that we can use the construction to find a $\tau \in \mathsf{mA}(\mathcal{A})$, whenever such a τ exists.

Outline of the proof. The proof is a direct consequence of Lems. 1 to 3 below. We observe that on the left-hand side of Thm. 1, the monitor, horizon, observations, and risk do not occur, they must be encoded into the colored MDP. Furthermore,

[3] Colored MDPs with color-consistent policies coincide with memoryless policies for partially observable MDPs. However, POMDPs often consider history-dependent (belief-based) policies. We use *colored MDPs* to avoid any confusion.

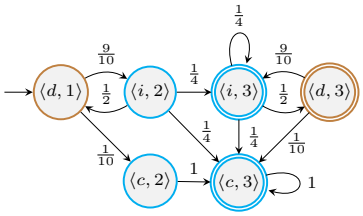


Fig. 4: The HMM for example 4. States are named by the HMM state, $\{d, i, c\}$ and the monitor state, $\{1, 2, 3\}$. The alarm states are marked accepting.

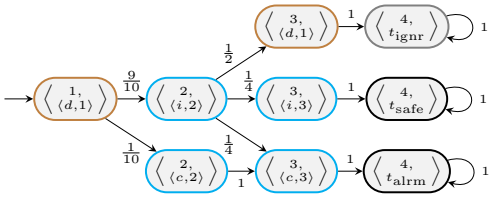


Fig. 5: The HMM for example 5, brown and cyan are dry and icy observations. Black is the new z_{end} observation, and gray is the new z_{ignore} observation. All states are named with the step i, model state s and monitor state j as $\langle i, \langle s, j\rangle\rangle$.

while missed alarms are defined using conditional probabilities, the policy synthesis problem is over reachability probabilities. We describe our transformation in several steps. In Sec. 3.2, we encode the monitor into the HMM and transform the HMM to include both the horizon and the risk. In Sec. 3.3, we resolve the conditioning and replace observations by nondeterminism.

Corollary 2. *There exists a map $t(\sigma) = \tau$, which, given a color-consistent policy σ, finds its associated trace τ.*

3.2 The (Acyclic) Conditional Trace Risk Problem

First, we show how asking for a missed alarm can be rephrased into the conceptually simpler *conditional trace risk* (CTR) problem. We will further simplify the problem such that we are left with a problem on acyclic HMMs.

CTR problem We build (a mild variation of) a standard product construction [15] between HMM and the DFA. We define the *alarm states* T as those states which correspond to non-accepting states in the DFA. This is equivalent to taking a product with the complement of the monitor.

Example 4. Fig. 4 shows the product of the HMM and monitor $\mathcal{B}$ from Fig. 3. Starting in the initial states d and 1, the HMM transitions to the i state with probability $9/10$. This is an *icy* state, and thus the monitor takes the *icy* transition to state 2. In the product, this is a transition from $\langle d, 1\rangle$ to $\langle i, 2\rangle$ with probability $9/10$. The alarm states are any product states of the form $\langle _, 3\rangle$.

Definition 9 (HMM product). *Given an HMM $\mathcal{M} = (S, \iota^{\mathcal{M}}, \mathbf{P}, Z, obs, r)$ and monitor $\mathcal{A} = (Q, \Sigma, \delta, \iota^{\mathcal{A}}, F)$, the* product HMM *$\langle \mathcal{M}_{\times\mathcal{A}}, T\rangle$ is the HMM $\mathcal{M}_{\times\mathcal{A}} := (S \times Q, \langle \iota^{\mathcal{M}}, \iota^{\mathcal{A}}\rangle, \mathbf{P}', Z, obs', r')$ with $obs'(\langle s, q\rangle) := obs(s)$, $r'(\langle s, q\rangle) := r(s)$, $\mathbf{P}'(\langle s, q\rangle, \langle s', \delta(q, obs(s'))\rangle) := \mathbf{P}(s, s')$ and $\mathbf{P}'(x, x') := 0$ otherwise, and finally the alarm states $T := S \times F$.*

In the product, we can find a trace τ whose conditional trace risk exceeds a threshold iff τ is a missed alarm. We state the decision problem that needs to be solved: It is key to our computational complexity analysis in Sec. 5.

Definition 10 (CTR Decision Problem). *Given HMM $\mathcal{M}$ with states S and risk r, horizon h, alarm states $T \subseteq S$, and threshold $\lambda_u \in \mathbb{R}_{\geq 0}$, decide if*

$$\exists \tau \in \mathcal{L}(\mathcal{M}).\ \sum\nolimits_{\pi \in \Pi^{\mathcal{M}}_{=h} \mid \pi_{\downarrow} \in T} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_{\downarrow}) > \lambda_u.$$

We denote the set of witnesses τ to a CTR decision problem as $\mathsf{CTR}(\mathcal{M}, h, T, \lambda_u)$. The following lemma states the correctness of the transformation. It follows directly from the definition of missing alarms and the product with the complement.

Lemma 1. *Using the notation from Thm. 1, Def. 9 and Def. 10:*

$$\exists \tau \in \mathsf{mA}^{=h}_{\mathcal{M},\lambda_u}(\mathcal{A}) \quad \textit{iff} \quad \exists \tau \in \mathsf{CTR}(\mathcal{M}_{\times \overline{\mathcal{A}}}, h, T, \lambda_u)$$

ACTR problem We further simplify the problem by unrolling the model along the horizon h and eliminating the risk function. For the first h steps, the unrolling is standard. At the horizon we transition to three dedicated states $\langle h+1, t_{\text{alrm}}\rangle$, $\langle h+1, t_{\text{safe}}\rangle$ and $\langle h+1, t_{\text{ignr}}\rangle$[4] according to the risk and alarm states T.

Example 5. Consider the HMM $\mathcal{M}$ depicted in Fig. 4, which was defined in example 4. We unroll $\mathcal{M}$ with a horizon $h = 3$. This yields the HMM in Fig. 5. The states are tuples of the unrolled step and the CTR state from $\mathcal{M}$. The initial state becomes $\langle 1, \langle d, 1\rangle\rangle$, which corresponds to a state at step 1 and $\langle d, 1\rangle$ from the $\mathcal{M}$. This state transitions to $\langle 2, \langle i, 2\rangle\rangle$ and $\langle 2, \langle c, 2\rangle\rangle$. States with a step value equal to the horizon are treated differently. From states at the horizon, we transition to either of three states that help classify the trace. Consider $\langle 3, \langle i, 3\rangle\rangle$, it is at the horizon $h = 3$, and it is in T. We want to classify such terminal states. As the (normalized) risk of $\langle 3, \langle i, 3\rangle\rangle$ is 0, the state is safe and $\langle 3, \langle i, 3\rangle\rangle$ transitions with probability 1 to $\langle 4, t_{\text{safe}}\rangle$. Next, Consider $\langle 3, \langle d, 1\rangle\rangle$, $\langle d, 1\rangle$, it is not in T, but the state is at the horizon. This state can be ignored, and we transition to the ignore state, $\langle 4, t_{\text{ignr}}\rangle$.

Definition 11 (Unrolling with risk). *The (risk-)unrolled HMM $\mathcal{M}_{\blacktriangleright h}$ of an HMM $\mathcal{M} = (S, \iota, \mathbf{P}, Z, obs, r)$ with alarm states T and horizon h is the HMM*

$$\mathcal{M}_{\blacktriangleright h} := (\{1, \ldots, h\} \times S \cup S_{\mathsf{end}}, \langle 1, \iota\rangle, \mathbf{P}', Z \cup \{z_{end}, z_{ignore}\}, obs', r'),$$

with $S_{\mathsf{end}} := \{\langle h+1, t_{alrm}\rangle, \langle h+1, t_{safe}\rangle, \langle h+1, t_{ignr}\rangle\}$, $r'(\cdot) = 0$, obs' given by (1) $obs'(\langle i, s\rangle) := obs(s)$, (2) $obs'(\langle h+1, t_{ignr}\rangle) := z_{ignore}$, and (3)

[4] We add the $\langle h+1, t_{\text{ignr}}\rangle$ state to easily modify the transformation for the no-false-alarms problem (See Sec. 3.4).

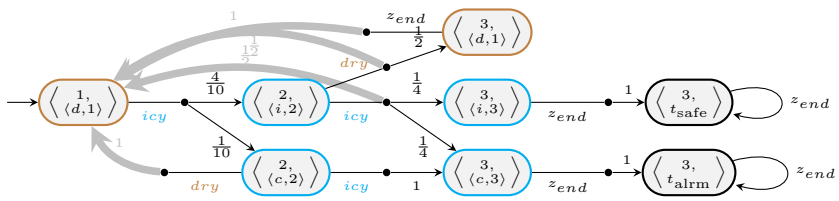


Fig. 6: The MDP from example 6. Unreachable states are omitted, as are any actions which return to the initial state from every state at the same step.

$$obs'(\langle h+1, t_{alrm}\rangle) = obs'(\langle h+1, t_{safe}\rangle) := z_{end}, \text{ and } \mathbf{P}' \text{ given by:}$$

$$\forall_{i\in\{1,\dots,h-1\}} \quad \mathbf{P}'(\langle i,s\rangle, \langle i+1,s'\rangle) := \mathbf{P}(s,s'),$$

$$\mathbf{P}'(\langle h,s\rangle, \langle h+1,t\rangle) := \begin{cases} \frac{r(s)}{\max_{s\in S} r(s)} & \text{if } s\in F \text{ and } t = t_{alrm}, \\ 1 - \frac{r(s)}{\max_{s\in S} r(s)} & \text{if } s\in F \text{ and } t = t_{safe}, \\ 1 & \text{if } s\notin F \text{ and } t = t_{ignr}, \\ 0 & \text{otherwise,} \end{cases}$$

$$\mathbf{P}'\left(\langle h+1,t\rangle, \langle h+1,t\rangle\right) := 1.$$

Lemma 2. *Given an HMM $\mathcal{M}$, horizon h, alarm states T, and threshold λ_u, there exists a $\lambda \in (0,1]$ such that, using z_{end} and t_{alrm} from Def. 11:*

$$\exists \tau \in \mathsf{CTR}(\mathcal{M}, h, t, \lambda_u) \quad \textit{iff} \quad \exists \tau \in \mathcal{L}(\mathcal{M}_{\blacktriangleright h}). \sum_{\pi\in\Pi^{\mathcal{M}}} Pr^{\mathcal{M}}(\pi\cdot t_{alrm} \mid \tau\cdot z_{end}) \geq \lambda.$$

3.3 Reduction to Consistent Policy Synthesis

For Lemma 2, we must find a trace such that a conditional reachability probability exceeds a threshold. We reformulate this into a policy synthesis problem (Sec. 3.1). The transformation combines two ideas: First, in every state, the policy can select the next observation, loosely inspired by [14]. Second, we reformulate conditional reachability probabilities into reachability probabilities, as in [34,16].

Example 6. We transform the HMM from Fig. 5 to the colored MDP in Fig. 6. Consider $\langle 2, \langle i,2\rangle\rangle$ in the HMM. The next observation is either *dry* or *icy*, which corresponds to the two actions from $\langle 2, \langle i,2\rangle\rangle$ in the colored MDP. The *dry* action transitions to the ensuing states with the *dry* observation. The remaining probability of $1/2$ for the *dry* action is directed to the initial state. This is similarly applied to the *icy* action. State $\langle 2, \langle c,2\rangle\rangle$ does not reach any *dry* states, we still add a *dry* action redirecting to the initial state, as *all* states with the same step value must have the same actions. State $\langle 3, \langle d,1\rangle\rangle$ in the HMM only has a transition to an z_{ignore} state. We fully remove the z_{ignore} state and the z_{ignore} observation in the MDP. Finally, we make sure all states with step 3 have the same actions, which explains the z_{end} action from state $\langle 3, \langle d,1\rangle\rangle$.

Definition 12. *Given the HMM from Def. 11*

$$\mathcal{M}_{\blacktriangleright h} := (S' := \{1, \ldots, h\} \times S \cup S_{\mathsf{end}}, \langle 1, \iota \rangle, \mathbf{P}, Z' := Z \cup \{z_{end}, z_{ignore}\}, obs, r),$$

we define the colored MDP $\mathcal{M}_{\geqslant h} := ((S' \setminus \{\langle h+1, t_{ignr} \rangle\}, \iota, Act, \mathbf{P}'), C, c)$ *with* $Act := Z' \setminus \{z_{ignore}\}$, $C := \{1, \ldots, h\}$, *coloring* c *s.t.* $c(\langle i, s \rangle) := i$, *and*

$$\mathbf{P}'(\langle i, s \rangle, z, \langle j, q \rangle) := \begin{cases} \mathbf{P}(\langle i, s \rangle, \langle j, q \rangle) & \text{if } obs(\langle j, q \rangle) = z, \\ \sum_{(\langle k, q' \rangle) \in S \mid obs(\langle k, q' \rangle) \neq z} \mathbf{P}(\langle i, s \rangle, \langle k, q' \rangle) & \text{if } (\langle j, q \rangle) = \iota, \\ 0 & \text{otherwise} \end{cases}$$

Thus, we transition normally to a state if the action and observation of the target state are equal, otherwise we set the transition probability to zero. All the remaining probability mass is redirected towards the initial state[5].

The above construction allows for conditioning on a trace τ by constructing a policy σ that selects the ith observation of τ in the state with step i.

Definition 13 (Trace consistent policy). *Given a colored MDP* $\mathcal{M}_{\geqslant h}$ *as in Def. 12 and a trace* $\tau \in (Z')^\star$, *a* trace consistent policy *satisfies* $\sigma_\tau(\langle i, s \rangle) := \tau^{(i)}$, *where* $\tau^{(i)}$ *is the* i*th observation in* τ, *for* $i \leq |\tau|$ *and* $\sigma_\tau(\langle i, s \rangle) := z_{end}$ *otherwise.*

Using the coloring as described in Def. 12, the trace consistent policies coincide with color-consistent policies (Def. 8). Finding a missed-alarm trace now reduces to solving the color-consistent policy synthesis problem for targets $\{\langle h+1, t_{\text{alrm}} \rangle\}$.

Lemma 3. *Given an HMM* $\mathcal{M}$, *horizon* h *alarm states* T, *and threshold* λ, *it holds that*

$$\exists \tau \in \mathcal{L}(\mathcal{M}_{\blacktriangleright h}). \sum_{\pi \in \Pi^{\mathcal{M}_{\blacktriangleright h}}} Pr^{\mathcal{M}_{\blacktriangleright h}}(\pi \cdot \langle h+1, t_{alrm} \rangle \mid \tau \cdot z_{end}) \geq \lambda$$
$$\Updownarrow$$
$$\exists \sigma \in \Sigma_c. Pr_\sigma^{\mathcal{M}_{\geqslant h}}(\Diamond\{\langle h+1, t_{alrm} \rangle\}) \geq \lambda$$

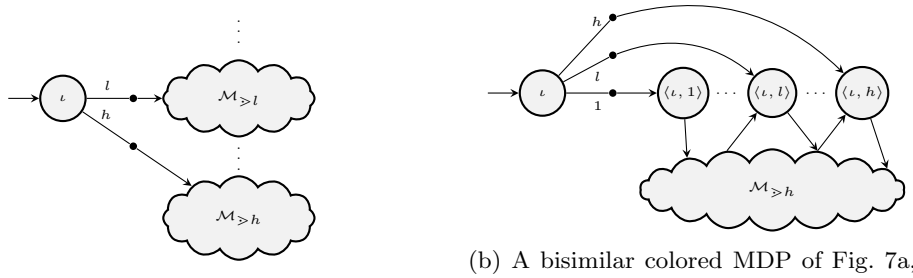
Where σ *is trace consistent for* τ.

Note that the colored part of the colored MDPs is absolutely necessary in order to map a policy of the colored MDP to a trace.

Example 7. Consider the MDP from Fig. 6 while ignoring the coloring. A function mapping $\langle 2, \langle i, 2 \rangle \rangle$ to *icy* and $\langle 2, \langle c, 2 \rangle \rangle$ to *dry* is a valid policy for the MDP. However, there does not exist a trace which describes the actions taken in the model, i.e., the trace starts with *icy* but then sometime contains *icy* and sometimes *dry*.

Restricting our policies to color-consistent policies makes sure in every step of the model we only ever take one action, corresponding with one observation in the trace at that step.

[5] In the implementation, we can prune actions where, from every state with the same color, the action redirects to the initial state.



(a) A colored MDP combining the colored MDPs from Def. 12 for horizons up to h, $\{\mathcal{M}_{\geqslant l} \mid l \in \{1, \ldots, h\}\}$, into one MDP.

(b) A bisimilar colored MDP of Fig. 7a, containing only $\mathcal{M}_{\geqslant h}$. States $\langle \iota, 1 \rangle$ through $\langle \iota, h \rangle$ from $\mathcal{M}_{\geqslant h}$ are shown separate in order to show their relation to ι.

Fig. 7: Transformation steps needed for Thm. 2.

3.4 Adapting to No-False-Alarms and Smaller Traces

Traces of length at most the horizon. The approach for Thm. 1 only works for traces of length exactly equal to the horizon. We generalize this approach to traces of length at most the horizon.

Theorem 2. *Given an HMM $\mathcal{M}$, a monitor $\mathcal{A}$, unsafe threshold λ_u, horizon h, and risk r, there is a colored MDP $\mathcal{M}^C$ with target states T, and threshold λ s.t.*

$$\exists \sigma \in \Sigma_c.\ Pr_{\sigma}^{\mathcal{M}^C}(\Diamond T) \geq \lambda \quad \textit{iff} \quad \exists \tau \in \mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}).$$

The main insight for this theorem is shown in Fig. 7. We combine the colored MDPs given by Thm. 1 for horizons 1 to h into one colored MDP, such that a policy starts by choosing which length trace to use (Fig. 7a). We can also instead directly construct a bisimulation quotient $\mathcal{M}_{\geqslant h}$ of this combined MDP with a small addition (Fig. 7b). We detail this construction in [35, App. A].

Finding false alarms (solving Prob. 2). We modify the transformation from Thm. 2 such that it solves the no-false-alarms problem. This problem differs in two ways from the no-missed-alarms problem. We are finding a trace *accepted by* the monitor, and we find a trace whose risk is *below* the *safe threshold*.

Theorem 3. *Given an HMM $\mathcal{M}$, a monitor $\mathcal{A}$, safe threshold λ_s, horizon h, and risk r, there is a colored MDP $\mathcal{M}^C$ with target states T, and threshold λ s.t.*

$$\exists \sigma \in \Sigma_c.\ Pr_{\sigma}^{\mathcal{M}^C}(\Diamond T) \geq \lambda \quad \textit{iff} \quad \exists \tau \in \mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}).$$

We highlight the ideas here, for details see [35, App. A]. In order to find a trace accepted by the monitor, we no longer take the complement of the monitor while transforming to CTR. To find a safe trace we compute reachability on $\langle h+1, t_{\text{safe}} \rangle$ instead of $\langle h+1, t_{\text{alrm}} \rangle$ while taking as a threshold $1 - \lambda$. Thus, we find a trace whose probability of being safe is above a threshold[6].

[6] We cannot directly aim to compute a trace whose risk is below a threshold since minimizing reachability of $\langle h+1, t_{\text{alrm}} \rangle$ will result in a scheduler that always resets to

4 Learning Correct Monitors

We present *learning* correct monitors (Prob. 3), by combining automata learning using a *Minimally Adequate Teacher* (MAT, [11]) and monitor verification.

MAT framework. We briefly recap the MAT framework, for details see [47]. A minimally adequate teacher answers two types of questions: A *membership query* (MQ), which in our setup means *should a trace be accepted by the monitor?*, and an *equivalence query* (EQ), *is this monitor correct?* Furthermore, if the answer to an equivalence query is negative, we must provide a counterexample that witnesses why the monitor is not correct. Various algorithms implementing the MAT framework for DFA learning exist. For the purpose of this paper, we use the $L^\star$ algorithm [11] to learn a monitor. The learner asks MQs to the teacher until a *hypothesis monitor* can be constructed which is consistent with the MQs. Once such a hypothesis is constructed, its correctness is verified using an EQ.

Verification as a MAT. To learn a monitor $\mathcal{A}$, we provide the HMM $\mathcal{M}$, a risk function r, a horizon h, a learning threshold λ_l, and the safe and unsafe thresholds λ_s and λ_u. The additional learning threshold λ_l is used to define an MQ whenever $\lambda_s \neq \lambda_u$: In particular, for MQs, each trace must be unambiguously safe or unsafe: the MAT framework does not allow for flagging certain traces as *don't care*, while traces with a risk between λ_s and λ_u can be considered don't care in our setting. Likewise, the MQ must also be defined for traces $\tau \notin \mathcal{L}(\mathcal{M})$ or traces longer than the horizon. We thus adapt the notion of safe traces from Def. 4.

Definition 14. *Given any trace $\tau \in Z^\star$ and a horizon h, membership query MQ_{λ_l} is a function such that $\mathrm{MQ}_{\lambda_l}(\tau)$ is unsafe iff $\tau \in \mathcal{L}(\mathcal{M})$[7] and $\tau \in \mathbb{U}_{\lambda_l}^{\leq h}$.*

Such a function for MQ_{λ_l} can be defined by keeping track of the probability of being in each state after every observation from the trace *or* by model checking the induced Markov chain that reflects the trace-consistent policy in Sec. 3 [39,34]. For EQs, we simply use the notion of correctness from Def. 6.

Definition 15. *Given an HMM $\mathcal{M}$, and a monitor $\mathcal{A}$, an $\mathrm{EQ}_{\lambda_s,\lambda_u}$ is a function $\mathrm{EQ}_{\lambda_s,\lambda_u}(\mathcal{A}) \in \{\top\} \cup Z^\star$, such that, $\mathrm{EQ}^{\mathcal{M}}_{\lambda_s,\lambda_u}(\mathcal{A})$ holds if $\mathcal{A}$ is correct for $\mathcal{M}$ with λ_s, and λ_u (in the sense of Def. 6), and $\mathrm{EQ}^{\mathcal{M}}_{\lambda_s,\lambda_u}(\mathcal{A})$ returns a missed alarm or false alarm trace for an incorrect $\mathcal{A}$.*

The EQ requires checking both for no-missing-alarms, and for no-false-alarms. We perform these checks using our novel verification algorithm described in Sec. 3.

Lemma 4. *Given a MAT with a $\mathrm{EQ}_{\lambda_s,\lambda_u}$ and a MQ_{λ_l}, a monitor learned with $L^\star$ is correct as long as $\lambda_s \leq \lambda_l \leq \lambda_u$.*

---

the initial state if possible. Consider the colored MDP in Fig. 6, the color-consistent policy with the lowest probability of reaching the alarm state is as follows: take the *dry* action in step 2, and z_{end} in step 3. This policy has probability zero of reaching either $\langle h+1, t_{\mathrm{alrm}}\rangle$ or $\langle h+1, t_{\mathrm{safe}}\rangle$, and thus does not map to a valid trace.

[7] Defining traces $\tau \notin \mathcal{L}(\mathcal{M})$ as safe is an arbitrary design decision.

When $\lambda_s < \lambda_u$, the EQ has an inconclusive area given by the interval (λ_s, λ_u). This means that our EQ does not check for equivalence, but simply accepts any correct monitor [37]. We investigate the effect of this inconclusive area in Sec. 6.

Conformance queries. An alternative to the EQ in Def. 15 is a conformance query [26]. It tests a monitor by sampling traces from the HMM and checking if the MQ and the monitor agree. If the monitor and the MQ don't agree on a trace, it is given as a counterexample. In our approach we use a hybrid of conformance and verification EQs. Monitors produced early in the learning process often contain many missed alarms and false alarms. Verification can find them, however, applying the transformation from Sec. 3 has a constant cost. Conformance queries can often find a counterexample faster if they have a high probability of occurring. We thus first try to find easy counterexamples using conformance, when this does not produce any we perform our verification routine.

5 Computational Complexity

This section discusses the hardness of monitor verification (Thm. 4) and the inapproximability of a related optimization problem (Lem. 6).

Theorem 4. Is a monitor correct? *(w. unary coded horizon) is coNP-complete.*

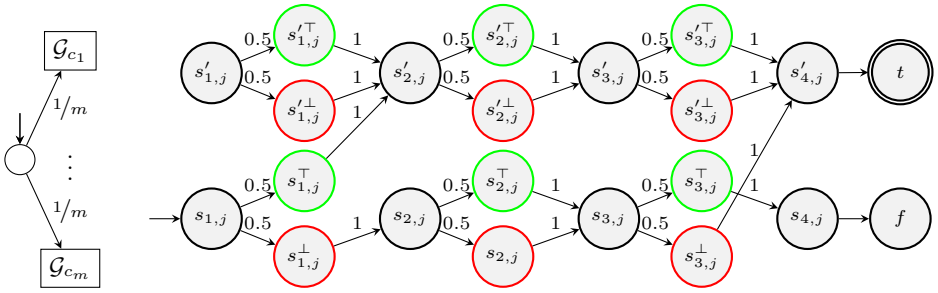
In fact, we study the dual to this problem, i.e., checking the existence of a counterexample. We call this problem *monitor co-verification*. For monitor co-verification, *membership* in NP follows from false alarms or missed alarms (of length up to horizon) being the witnesses. Verifying whether a trace is a false or missed alarm can be done in polynomial time, by checking whether the automaton accepts it and computing the trace risk (see Sec. 4).

To establish NP-hardness, we consider the CTR problem from Def. 10. As a solution to the monitor co-verification problem solves the CTR problem (using a trivial monitor), this implies NP-hardness of the former problem.

Lemma 5. *The CTR Decision Problem is (strongly) NP-hard.*

The proof features a reduction from CNF-SAT, the problem of satisfiability of a propositional formula. We illustrate the reduction, details are in [35, App. B].

We construct HMM $\mathcal{M}_\varphi$ from CNF φ over variables X such that there is a trace with risk 1 iff there is a satisfying assignment to φ. In particular, that trace exists iff there is a trace τ s.t. all corresponding paths π with $obs(\pi) = \tau$ reach state t. The traces are of the form $\#\# \cdot \{\bot, \top\} \cdot \# \cdot \{\bot, \top\} \cdots \#$: Trace $\#\# \cdot \alpha(x_0) \cdot \# \cdot \alpha(x_1) \cdots \#$ represents an assignment $\alpha \colon X \to \{\bot, \top\}$. We construct $\mathcal{M}_\varphi$ such that any trace that ensures reaching t with probability one encodes a satisfying assignment for φ. We create gadgets for every clause. The gadgets are connected as in Fig. 8a: That is, to ensure reaching t along every path, we must reach t in every gadget. The gadget $\mathcal{G}_j$ intuitively 'evaluates' c_j with respect to an assignment, as exemplified in Fig. 8b. A path (or its trace) through $\mathcal{G}_j$ 'reads' variable x_i in state $s_{i,j}$ and transitions to $s^\top_{i,j}$ or to $s^\bot_{i,j}$. The states are labelled $\#, \top, \bot$, respectively. However, for a trace where $\alpha(x_i) = \top$,



(a) HMM. (b) Gadget $\mathcal{G}_c$ demonstrated for $c = x_1 \vee \neg x_3$

Fig. 8: Illustrations for Lemma 5, with m clauses and variables x_1, x_2, x_3.

only the former path corresponds to the trace (and symmetrically for $\alpha(x_i) = \bot$). That path reaches state t iff the assignment satisfies at least one literal in the clause.

We now show that the construction above suffices to show that it is hard to approximate the maximal risk that a monitor admits.

Definition 16 (CTR Optimization Problem). *Given an HMM $\mathcal{M}$ with states in S, a unary encoded horizon h, a set of alarm states $T \subseteq S$:*

$$\max_{\tau \in \mathcal{L}(\mathcal{M})} \sum_{\pi \in \Pi^{\mathcal{M}}_{=h} \mid \pi_{\downarrow} \in T} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_{\downarrow}).$$

Lemma 6. *The CTR Optimization Problem is APX-hard.*

This follows from a strict reduction from MAX-3SAT, which is an inapproximable and APX-hard problem [28]. The construction coincides with the reduction in Lem. 5 by observing that the conditional probability to reach a t state is given by $1/m$ times the number of satisfied clauses, i.e., we can compute the maximal number of satisfied clauses in φ using the HMM $\mathcal{M}_{\varphi}$.

6 Experiments

We empirically evaluate the monitor verification (Sec. 3) and monitor learning (Sec. 4) algorithms using our prototype implementation called ToVer. Code, benchmarks, and logs will be publicly available via the artifact evaluation.

Setup. The ToVer tool is implemented in Python and C++ on top of the model checker Storm [30] for data structures and for the MQs in Sec. 4 [34]. We use PAYNT [9] to verify colored MDPs (Def. 7), using exact arithmetic to avoid numerical problems on these types of benchmarks [27]. The learner uses the AAlpy framework [36]. All experiments are run on a single thread of an AMD Ryzen TRP 5965WX, with a memory limit of 15 GiB, and a time-out of 12 hours.

Benchmarks. We take the benchmarks Airport, Refuel, Evade, and Hidden-Incentive from [34], adapted to HMMs. We add Icy-Driving, a scaled-up

		Benchmark								ToVer				
		h	MA/FA	$\lvert S^{\mathcal{M}}\rvert$	$\lvert \mathbf{P}^{\mathcal{M}}\rvert$	$\lvert Z\rvert$	$\lvert S^{\mathcal{A}}\rvert$	$\lvert \mathbf{P}^{\mathcal{A}}\rvert$	$\lvert \mathcal{L}^{\leq h}\rvert$	Time (s)	Trans (s)	PAYNT (s)	$\lvert \mathcal{M}_{\gg h}\rvert$	λ^{found}
AirportB-7	A-180	9	MA	470	2550	50	66	3300	10^{12}	5	1	4	2275	0.69
AirportB-7	A-182	9	FA	470	2550	50	66	3300	10^{12}	3	$\leq 1s$	3	2275	0.10
AirportB-7	A-184	9	MA	470	2550	50	319	15950	10^{12}	23	1	22	3503	0.41
AirportB-7	A-186	9	FA	470	2550	50	319	15950	10^{12}	17	1	15	3503	0.24
AirportB-7	A-188	9	MA	470	2550	50	569	28450	10^{12}	367	2	365	5111	✓
AirportB-7	A-190	9	FA	470	2550	50	569	28450	10^{12}	340	3	338	5111	✓
Evade	E-40	9	MA	385	1473	325	197	64025	10^{14}	34	26	4	680	1.00
Evade	E-42	9	FA	385	1473	325	197	64025	10^{14}	25	23	2	680	✓
Hidden-Incen.	H-2	10	FA	397	1649	100	106	10600	10^{18}	10	3	6	1261	0.30
Hidden-Incen.	H-10	10	FA	397	1649	100	298	29800	10^{18}	2972	4	2968	1411	✓
Icy-Driving	I-34	3	FA	3	6	2	8	16	10^{0}	$\leq 1s$	$\leq 1s$	$\leq 1s$	8	✓
Icy-Driving	I-10	10	FA	3	6	2	2	4	10^{4}	$\leq 1s$	$\leq 1s$	$\leq 1s$	29	✓
Icy-Driving	I-22	25	FA	3	6	2	2	4	10^{11}	1126	$\leq 1s$	1126	74	✓
RefuelB	R-20	10	MA	263	7127	72	31	2232	10^{22}	3	1	2	1812	✓
RefuelB	R-22	10	FA	263	7127	72	31	2232	10^{21}	5	2	3	1812	✓
SnL	S-76	16	MA	101	502	4	427	1708	10^{11}	393	$\leq 1s$	392	13547	0.54
SnL	S-78	16	FA	101	502	4	427	1708	10^{10}	1418	$\leq 1s$	1417	13547	0.00
SnL	S-80	16	MA	101	502	4	588	2352	10^{11}	4601	2	4600	14381	✓
SnL	S-82	16	FA	101	502	4	588	2352	10^{10}	2414	1	2413	14381	✓

Table 1: Subset of verification results found in [35, App. C.2]. The columns give the family name, an ID, horizon, and whether we check for missed alarms or false alarms. We give the size of the HMM (states, transitions), the number of observations, the size of the DFA (states, transitions), and the size of the language after pruning unreachable states. Furthermore, we list the run time for the complete verification procedure as well as the time spent on transforming the problem into a policy synthesis problem and the policy synthesis in PAYNT. Lastly, we list the size of the colored MDP produced by the transformation and the risk of the found counterexample. If no trace was found with a risk above (or below, for FA) the indicated threshold, the monitor is correct (✓).

version of the running example and SnL based on the game "Snakes and Ladders". While the benchmarks from the literature contain many observations, i.e., few states share an observation, the new benchmarks only have a few different observations. All benchmarks are scalable. The risk function is defined by a temporal property, e.g., the probability of reaching a bad state within a few steps.

Efficiency of Monitor Verification We first investigate scalability along different dimensions and identify the bottlenecks of our verification method.

Setup. We verify the HMMs with respect to three monitors obtained during the learning experiments (below, with $\lambda_s = \lambda_u = 0.3$). Every version of the benchmark is run on the first (incorrect) monitor that passed a limited conformance check, an (incorrect) monitor obtained halfway through the learning process, and the final correct monitor. We verify correctness w.r.t. the same λ_s, λ_u.

Results. We present our results in table 1, which is a subset of the 516 benchmarks shown in [35, App. C.2]. Generally, we observe that we verify the correctness of monitors on at least billions of traces, which shows that enumerating the traces is not a feasible alternative. Our verification handles monitors and HMMs with both

hundreds of states and up to hundred thousands of transitions, see benchmarks E-40, E-42, H-10. Benchmarks A-180, A-182 reflect verification w.r.t. almost trivial monitors, for which it is typically easy to find a counterexample, A-184, A-186, S-76, S-78 reflect a semi-correct monitor, and A-188, A-190, S-80, S-82 reflect verification of the same HMM with respect to a larger correct monitor. Increasing the horizon significantly increases the runtime, even for small models, e.g., I-34 compared to I-10 and I-22. In all benchmarks, the runtime consists almost exclusively of creating the input to PAYNT (taking the product and creating the MDP) and in running PAYNT. The former runs in polynomial time in the size of the input (see [35, App. C.1]), whereas the latter uses various heuristics to avoid the exponential computation time. In the current implementation, except for the (comparably) large EVADE benchmarks, the vast majority of time is spent on PAYNT. The transformation is never the bottleneck (see [35, App. C.2]).

Efficiency of Monitor Learning A key contribution of this paper is the ability to use verification for the EQs in monitor learning. We consider the necessity of these EQs, the size of the learned monitors, and the efficiency of learning them, both for $\lambda_s = \lambda_u$ and $\lambda_s \neq \lambda_u$.

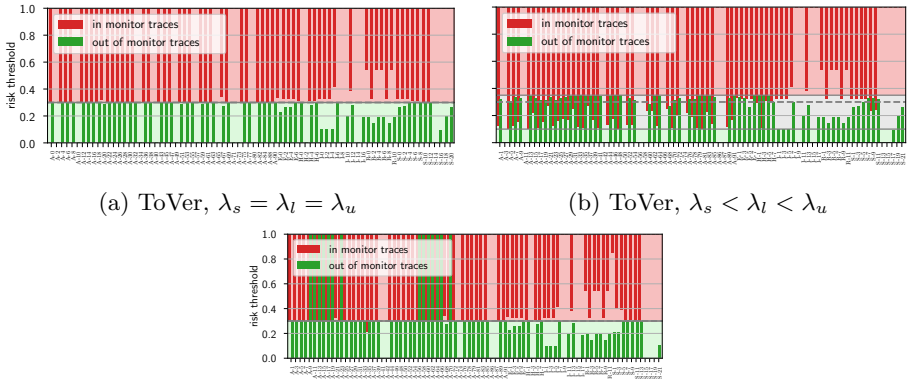
Setup. We use the MAT framework from Sec. 4. Before every EQ, we run conformance checking (max. 100 samples using as threshold λ_s and 100 samples with λ_u, see Sec. 4). As hyper-parameters, we investigate (1) $\lambda_s = 0.1, \lambda_l = 0.3, \lambda_u = 0.35$, with inconclusive trace, and (2) $\lambda_s = \lambda_l = \lambda_u = 0.3$, without inconclusive traces[8]. We compare against a baseline that does not use EQs, i.e., the baseline uses the MAT framework with only conformance checking (max. 100000 samples, different numbers of samples are tested in [35, App. D]).

Are the monitors correct? Using verification in the EQ, we always learn correct monitors. We validate this experimentally *and* show that the baseline does not always yield correct monitors. For every monitor we determine the unsafe trace with the lowest risk (*actual alarm threshold*, $\lambda_u^{\min}$) and the safe trace with the highest risk (*actual no-alarm threshold*, $\lambda_s^{\max}$). In a correct monitor, we have $\lambda_u^{\min} \geq \lambda_u$ and $\lambda_s^{\max} \leq \lambda_s$. Figs. 9a to 9c show $\lambda_u^{\min}$ and $\lambda_s^{\max}$ for ToVer and for the baseline. Visually, a monitor is correct if its red bar never touches the green area and the green bar never touches the red area. In 14 out of 87 benchmarks the baseline learns a monitor that misses alarms. One baseline learned monitor has false alarms.

How big are the monitors? ToVer learns monitors with hundreds of states and tens of thousands transitions, see Figs. 10a and 10b (log-scale!) and [35, App. E.2]. For the literature on AAL, these are large automata [49,2,44]. Comparing the sizes of the monitors learned using ToVer and the baseline, monitors are smaller (up to 8 times, mostly at least 1.5 times smaller)[9].

[8] We study correctness of monitors learned by the baseline w.r.t. different λ_l in [35, App. D].

[9] For $\lambda_s = \lambda_u$, the language of correct monitors learned with ToVer and baseline are equivalent up to the horizon, but the monitors respond differently on longer traces.



(a) ToVer, $\lambda_s = \lambda_l = \lambda_u$

(b) ToVer, $\lambda_s < \lambda_l < \lambda_u$

(c) Baseline

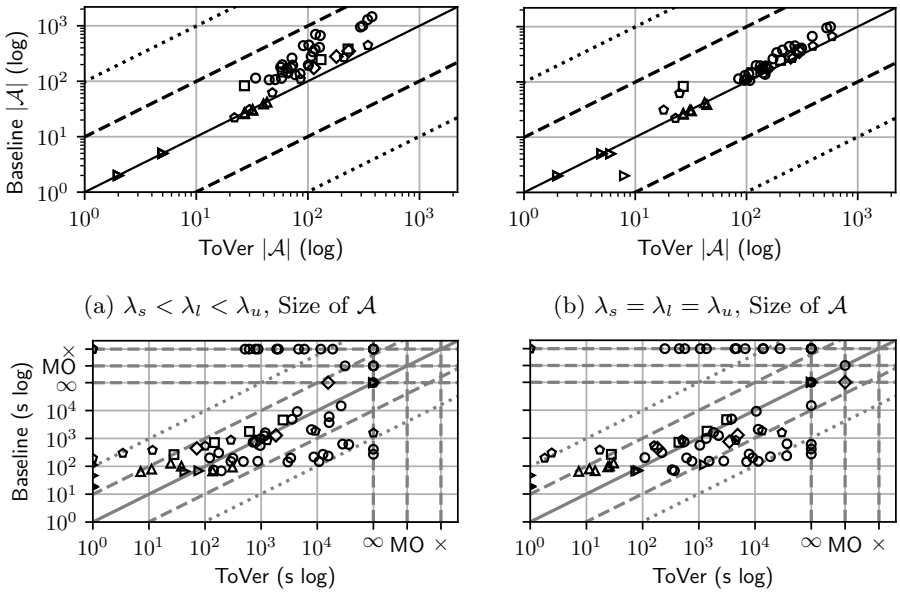
Fig. 9: Actual alarm and actual no-alarm thresholds from monitors learned with ToVer and baseline. The line between the green/gray is λ_u, the line between red/gray area is λ_s. The dotted line is λ_l. Missing bars reflect either time-outs or out-of-memory.

How fast do we learn the monitors? We compare the runtime of ToVer and baseline in the Figs. 10c and 10d (log-scale!). We remark that only ToVer is guaranteed to be correct. Neither of the two learning algorithms is clearly faster than the other, but ToVer has the potential to significantly accelerate the learning process, despite the high complexity. One reason could be that ToVer needs half or fewer EQs to learn a monitor as can be seen in [35, App. E.2]. In Fig. 11, we detail where the time is spent. For most benchmarks, the EQ (in particular, PAYNT) is the bottleneck. However, for several Evade benchmarks, most time is spent within $L^\star$ code. We conjecture this happens as finding counterexamples is simple in these models.

The role of an inconclusive area. We compare between $\lambda_s < \lambda_l < \lambda_u$ and $\lambda_s = \lambda_l = \lambda_u$, i.e., with and without an inconclusive area. The baseline does not actively support such an inconclusive area. With an inconclusive area, more monitors are correct (i.e., strictly speaking, we do not test equivalence but acceptance). The learner indeed finds monitors that are two to five times smaller (also compare Figs. 10a and 10b). For benchmarks Icy-Driving, Evade, and Airport, this also translates to faster runtimes than using conformance checking, sometimes by orders of magnitudes.

7 Related Work

This work studies *monitoring based on stochastic systems* and combines *active learning* with *probabilistic verification*. We consider related work in those directions.



(a) $\lambda_s < \lambda_l < \lambda_u$, Size of $\mathcal{A}$

(b) $\lambda_s = \lambda_l = \lambda_u$, Size of $\mathcal{A}$

(c) $\lambda_s < \lambda_l < \lambda_u$, Runtime

(d) $\lambda_s = \lambda_l = \lambda_u$, Runtime

Fig. 10: States in learned monitors and runtimes: ToVer vs. baseline. The runtime includes labels for time-outs (∞), out-of-memory (MO), and incorrect ($\times$).

Model-based monitoring for stochastic systems. Runtime verification is a wide field, see [24,41,29] for surveys. We review work on *model-based* runtime monitoring for stochastic systems. In particular, using state estimation on HMMs to decide whether to raise an alarm given a particular trace has been investigated in [42,45,48], extended to hybrid models [43], models with nondeterminism [34] and randomly timed models [14]. We use these techniques to answer membership queries. The HMMs for runtime monitoring can be learned from a set of traces, see, e.g., [13,12] and more recently [23], where they find the state risks at design time using model checking and use state estimation for runtime verification. Related to runtime monitoring is runtime enforcement, in particular shielding [19,40,25,31]. Shielding is successful in fully observable models but less studied in partial observable settings, in [21], shields are computed for qualitative properties. Finally, in [1], a more general notion of correct monitors via linear time μ-calculus is investigated, while in [17] a notion of correct *predictors* is introduced. Both can be seen as generalizations of our definitions.

Learning monitors. Learning monitors has been advocated in, e.g., [20,38,46,50]. Closest to our setting is recent work in [33], which also uses state estimation for membership queries, but combines this with conformance queries and learns decision trees rather than automata. Crucially, by using conformance queries, the guarantees are significantly weaker, see also our experiments.

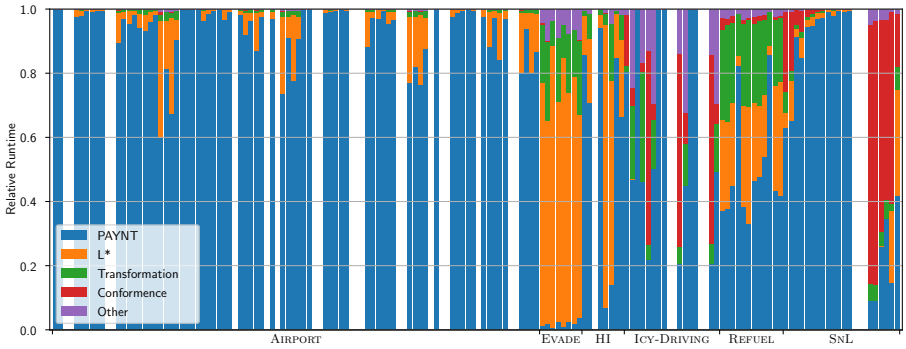


Fig. 11: Division of runtime for ToVer learning for different models. *PAYNT* is the time spent by PAYNT verifying colored MDPs. $L^\star$ is the time spent by the $L^\star$ algorithm creating a hypothesis including doing MQs. *Transformation* is the time spent transforming HMMs and monitors into colored MDPs for PAYNT. *Conformance* is the time spent on conformance testing during the EQs. Lastly, *Other* denotes any time not spend in the aforementioned processes. Empty bars denote HMMs for which learning did not finish.

Probabilistic verification. The verification of our monitors applies model checking of conditional probabilities [5,16] to runtime verification, similar to [34,14]. Most related is recent work in [14], where the models are CTMCs and the observation trace is uncertain itself. They also encounter a notion of trace-consistent policies, but instead of using synthesis, they overapproximate the verification by considering all policies. In contrast, our method is *complete*. Verification with partial observability as in our HMMs also occurs in the verification of partially observable MDPs [6], which can also be tackled using synthesis approaches [7]. Finally, considering MDPs as *distribution transformers* yields related but semantically different computationally hard problems that have been solved using (different) inductive synthesis approaches [3,4].

8 Conclusion and Future Work

This paper presented a first approach to verification of monitors with respect to hidden Markov models. It embeds this verification procedure in an automata learning framework. The empirical evaluation is encouraging but also shows the limitations of the off-the-shelf frameworks. We see three avenues for future work: (1) Dedicated synthesis methods for conditional probabilities and the specific structure of our colored MDPs. (2) Automata learning for acyclic models and don't-care results. (3) Faster verification for longer (or even unbounded) traces.

References

1. Aceto, L., Achilleos, A., Anastasiadi, E., Francalanza, A., Ingólfsdóttir, A., Lehtinen, K., Pedersen, M.R.: On probabilistic monitorability. In: Principles of Systems Design. Lecture Notes in Computer Science, vol. 13660, pp. 325–342. Springer (2022)
2. Aichernig, B.K., Tappler, M., Wallner, F.: Benchmarking combinations of learning and testing algorithms for automata learning. Formal Aspects Comput. **36**(1), 3:1–3:37 (2024)
3. Akshay, S., Chatterjee, K., Meggendorfer, T., Zikelic, D.: Mdps as distribution transformers: Affine invariant synthesis for safety objectives. In: CAV (3). Lecture Notes in Computer Science, vol. 13966, pp. 86–112. Springer (2023)
4. Akshay, S., Chatterjee, K., Meggendorfer, T., Zikelic, D.: Certified policy verification and synthesis for MDPs under distributional reach-avoidance properties. In: IJCAI. pp. 3–12. ijcai.org (2024)
5. Andrés, M.E., van Rossum, P.: Conditional probabilities over probabilistic and nondeterministic systems. In: TACAS. Lecture Notes in Computer Science, vol. 4963, pp. 157–172. Springer (2008)
6. Andriushchenko, R., Bork, A., Budde, C.E., Ceska, M., Grover, K., Hahn, E.M., Hartmanns, A., Israelsen, B., Jansen, N., Jeppson, J., Junges, S., Köhl, M.A., Könighofer, B., Kretínský, J., Meggendorfer, T., Parker, D., Pranger, S., Quatmann, T., Ruijters, E., Taylor, L., Volk, M., Weininger, M., Zhang, Z.: Tools at the frontiers of quantitative verification. CoRR **abs/2405.13583** (2024)
7. Andriushchenko, R., Bork, A., Ceska, M., Junges, S., Katoen, J., Macák, F.: Search and explore: Symbiotic policy synthesis in pomdps. In: CAV (3). Lecture Notes in Computer Science, vol. 13966, pp. 113–135. Springer (2023)
8. Andriushchenko, R., Ceska, M., Junges, S., Katoen, J.: Inductive synthesis of finite-state controllers for pomdps. In: UAI. Proceedings of Machine Learning Research, vol. 180, pp. 85–95. PMLR (2022)
9. Andriushchenko, R., Ceska, M., Junges, S., Katoen, J., Stupinský, S.: PAYNT: A tool for inductive synthesis of probabilistic programs. In: CAV (1). Lecture Notes in Computer Science, vol. 12759, pp. 856–869. Springer (2021)
10. Andriushchenko, R., Ceska, M., Macák, F., Junges, S., Katoen, J.: An oracle-guided approach to constrained policy synthesis under uncertainty. J. Artif. Intell. Res. **82**, 433–469 (2025)
11. Angluin, D.: Learning regular sets from queries and counterexamples. Inf. Comput. **75**(2), 87–106 (1987)
12. Babaee, R., Ganesh, V., Sedwards, S.: Accelerated learning of predictive runtime monitors for rare failure. In: RV. Lecture Notes in Computer Science, vol. 11757, pp. 111–128. Springer (2019)
13. Babaee, R., Gurfinkel, A., Fischmeister, S.: *P*revent : A predictive run-time verification framework using statistical learning. In: SEFM. Lecture Notes in Computer Science, vol. 10886, pp. 205–220. Springer (2018)
14. Badings, T.S., Volk, M., Junges, S., Stoelinga, M., Jansen, N.: Ctmcs with imprecisely timed observations. In: TACAS (2). Lecture Notes in Computer Science, vol. 14571, pp. 258–278. Springer (2024)
15. Baier, C., Katoen, J.: Principles of model checking. MIT Press (2008)
16. Baier, C., Klein, J., Klüppelholz, S., Märcker, S.: Computing conditional probabilities in markovian models efficiently. In: TACAS. Lecture Notes in Computer Science, vol. 8413, pp. 515–530. Springer (2014)

17. Baier, C., Klüppelholz, S., Piribauer, J., Ziemek, R.: Formal quality measures for predictors in markov decision processes. CoRR **abs/2412.11754** (2024)
18. Bartocci, E., Falcone, Y. (eds.): Lectures on Runtime Verification - Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457. Springer (2018)
19. Bloem, R., Könighofer, B., Könighofer, R., Wang, C.: Shield synthesis: Runtime enforcement for reactive systems. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 9035, pp. 533–548. Springer (2015)
20. Cairoli, F., Bortolussi, L., Paoletti, N.: Neural predictive monitoring under partial observability. In: RV. Lecture Notes in Computer Science, vol. 12974, pp. 121–141. Springer (2021)
21. Carr, S., Jansen, N., Junges, S., Topcu, U.: Safe reinforcement learning via shielding under partial observability. In: AAAI. pp. 14748–14756. AAAI Press (2023)
22. Chatterjee, K., Chmelik, M., Davies, J.: A symbolic sat-based algorithm for almost-sure reachability with small strategies in pomdps. In: AAAI. pp. 3225–3232. AAAI Press (2016)
23. Cleaveland, M., Sokolsky, O., Lee, I., Ruchkin, I.: Conservative safety monitors of stochastic dynamical systems. In: NFM. Lecture Notes in Computer Science, vol. 13903, pp. 140–156. Springer (2023)
24. Falcone, Y., Fernandez, J., Mounier, L.: What can you verify and enforce at runtime? International Journal on Software Tools for Technology Transfer **14**(3), 349–382 (2012)
25. Fulton, N., Platzer, A.: Safe reinforcement learning via formal methods: Toward safe control through proof and learning. In: AAAI. AAAI Press (2018)
26. Groce, A., Peled, D.A., Yannakakis, M.: Adaptive model checking. Log. J. IGPL **14**(5), 729–744 (2006)
27. Hartmanns, A., Junges, S., Quatmann, T., Weininger, M.: A practitioner's guide to MDP model checking algorithms. In: TACAS (1). Lecture Notes in Computer Science, vol. 13993, pp. 469–488. Springer (2023)
28. Håstad, J.: Some optimal inapproximability results. J. ACM **48**(4), 798–859 (2001)
29. Havelund, K., Reger, G., Rosu, G.: Runtime verification past experiences and future projections. In: Computing and Software Science, Lecture Notes in Computer Science, vol. 10000, pp. 532–562. Springer (2019)
30. Hensel, C., Junges, S., Katoen, J., Quatmann, T., Volk, M.: The probabilistic model checker storm. Int. J. Softw. Tools Technol. Transf. **24**(4), 589–610 (2022)
31. Jansen, N., Könighofer, B., Junges, S., Serban, A., Bloem, R.: Safe reinforcement learning using probabilistic shields (invited paper). In: International Conference on Concurrency Theory (CONCUR). LIPIcs, vol. 171, pp. 3:1–3:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2020)
32. Jha, S., Seshia, S.A.: A theory of formal synthesis via inductive learning. Acta Informatica **54**(7), 693–726 (2017)
33. Junges, S., Seshia, S.A., Torfah, H.: Active learning of runtime monitors under uncertainty. In: IFM. Lecture Notes in Computer Science, vol. 15234, pp. 297–306. Springer (2024)
34. Junges, S., Torfah, H., Seshia, S.A.: Runtime monitors for markov decision processes. In: CAV (2). Lecture Notes in Computer Science, vol. 12760, pp. 553–576. Springer (2021)
35. van der Maas, L., Junges, S.: Learning verified monitors for hidden markov models (2025), `https://arxiv.org/abs/2504.05963`, technical report of this paper.

36. Muskardin, E., Aichernig, B.K., Pill, I., Pferscher, A., Tappler, M.: Aalpy: an active automata learning library. Innov. Syst. Softw. Eng. **18**(3), 417–426 (2022)
37. Peled, D.A., Vardi, M.Y., Yannakakis, M.: Black box checking. J. Autom. Lang. Comb. **7**(2), 225–246 (2002)
38. Phan, D.T., Grosu, R., Jansen, N., Paoletti, N., Smolka, S.A., Stoller, S.D.: Neural simplex architecture. In: NFM. Lecture Notes in Computer Science, vol. 12229, pp. 97–114. Springer (2020)
39. Rabiner, L.R.: A tutorial on hidden markov models and selected applications in speech recognition. Proc. IEEE **77**(2), 257–286 (1989)
40. Ramadge, P.J., Wonham, W.M.: Supervisory control of a class of discrete event processes. SIAM Journal on Control and Optimization **25**(1), 206–230 (1987). `https://doi.org/10.1137/0325013`
41. Sánchez, C., Schneider, G., Ahrendt, W., Bartocci, E., Bianculli, D., Colombo, C., Falcone, Y., Francalanza, A., Krstic, S., Lourenço, J.M., Nickovic, D., Pace, G.J., Rufino, J., Signoles, J., Traytel, D., Weiss, A.: A survey of challenges for runtime verification from advanced application domains (beyond software). Formal Methods Syst. Des. **54**(3), 279–335 (2019)
42. Sistla, A.P., Srinivas, A.R.: Monitoring temporal properties of stochastic systems. In: VMCAI. Lecture Notes in Computer Science, vol. 4905, pp. 294–308. Springer (2008)
43. Sistla, A.P., Zefran, M., Feng, Y.: Runtime monitoring of stochastic cyber-physical systems with hybrid state. In: RV. Lecture Notes in Computer Science, vol. 7186, pp. 276–293. Springer (2011)
44. Smeenk, W., Moerman, J., Vaandrager, F.W., Jansen, D.N.: Applying automata learning to embedded control software. In: ICFEM. Lecture Notes in Computer Science, vol. 9407, pp. 67–83. Springer (2015)
45. Stoller, S.D., Bartocci, E., Seyster, J., Grosu, R., Havelund, K., Smolka, S.A., Zadok, E.: Runtime verification with state estimation. In: RV. Lecture Notes in Computer Science, vol. 7186, pp. 193–207. Springer (2011)
46. Torfah, H., Xie, C., Junges, S., Vazquez-Chanlatte, M., Seshia, S.A.: Learning monitorable operational design domains for assured autonomy. In: ATVA. Lecture Notes in Computer Science, vol. 13505, pp. 3–22. Springer (2022)
47. Vaandrager, F.W.: Model learning. Commun. ACM **60**(2), 86–95 (2017)
48. Wilcox, C.M., Williams, B.C.: Runtime verification of stochastic, faulty systems. In: RV. Lecture Notes in Computer Science, vol. 6418, pp. 452–459. Springer (2010)
49. Yang, N., Aslam, K., Schiffelers, R.R.H., Lensink, L., Hendriks, D., Cleophas, L., Serebrenik, A.: Improving model inference in industry by combining active and passive learning. In: SANER. pp. 253–263. IEEE (2019)
50. Zolfagharian, A., Abdellatif, M., Briand, L.C., S., R.: SMARLA: A safety monitoring approach for deep reinforcement learning agents. CoRR **abs/2308.02594** (2023)